Programmation impérative

Complément
Cahier

Licence informatique 2ème année

Université de La Rochelle



Ce document est distribué sous la licence CC-by-nc-nd

https://creativecommons.org/licenses/by-nc-nd/4.0/deed.fr

© 2011-2023 Christophe Demko
<christophe.demko@univ-lr.fr>

La Rochelle Université

2022-2023_1

Ce cahier fait suite au cahier de “Programmation impérative (basique)”

Les lignes commençant par $ représente une commande *unix*. Le $ ne fait pas partie de la commande proprement dite mais symbolise l'invite de commande.

Table des matières

1 Monômes univariés **3**

2 Polynômes univariés **5**

3 Itérateurs sur les polynômes **8**

4 Opérations sur les polynômes **10**

5 Polynômes multivariés **11**

Historique des modifications **12**

Table des figures

1 Représentation d'un monôme 3
2 Représentation d'un polynôme 6
3 Représentation d'un polynôme multivarié 11

Liste des exercices

1 Définition du type `Monomial` 3
2 Fonctions d'initialisation et de finalisation des monômes 3
3 Fonctions de création et de destruction des monômes 3
4 Fonctions de copie et de clonage des monômes 4
5 Fonctions d'accès aux données des monômes 4
6 Conversion en chaine de caractères des monômes 5
7 Écriture et lecture sur un fichier des monômes 5
8 Définition du type `Polynomial` 6
9 Fonctions d'initialisation et de finalisation des polynômes 6
10 Fonctions de création et de destruction des polynômes 6
11 Fonctions de copie et de clonage des polynômes 7
12 Fonctions d'accès aux données des polynômes 7
13 Fonction de conversion en chaine de caractères des polynômes 8
14 Écriture et lecture sur un fichier des polynômes 8
15 Définition du type `PolynomialIterator` 9
16 Fonctions de création et de destruction des itérateurs de polynômes 9
17 Fonctions d'itération sur les polynômes 9
18 Fonctions d'accès aux données sur les itérateurs de polynômes 10
19 Addition de polynômes 10

 © 2011-2023 Christophe Demko

20 Multiplication de polynômes 11
21 Revisiter les fonctions sur les polynômes multivariés 12

1 Monômes univariés

Un monôme [1] univarié est un monôme défini sur une seule variable. Ils seront représentés par une structure contenant un degré et une fraction.

Le monôme $-\frac{1}{8}x^5$ sera représenté par la figure 1

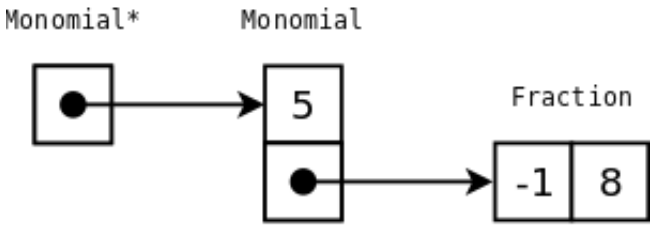


FIGURE 1 – Représentation d'un monôme



Exercice 1 *(Définition du type `Monomial`)*
Dans un nouveau projet, définissez le type `Monomial` permettant de stocker un monôme.

```
typedef struct _Monomial Monomial;
```

Faites en sorte d'encapsuler la structure dans un fichier `monomial.inc`. Il ne doit pas y avoir de structures dans le fichier `monomial.h`.



Exercice 2 *(Fonctions d'initialisation et de finalisation des monômes)*

1. Écrivez deux fonctions permettant d'initialiser (resp. de finaliser) la librairie gérant les monômes.

   ```
   extern bool monomial_init(void);
   extern bool monomial_finish(void);
   ```

   À son premier appel, la fonction d'initialisation doit appeler la fonction d'initialisation déjà définie dans les fractions `fraction_init`. À son dernier appel, la fonction de finalisation doit appeler la fonction de finalisation déjà définie dans les fractions `fraction_finish`. Les fonctions doivent partager un compteur comme cela a été le cas pour les fonctions `fraction_init` et `fraction_finish`.
2. Écrivez un programme de test `test-init-finish` permettant de valider les 2 fonctions écrites.



Exercice 3 *(Fonctions de création et de destruction des monômes)*

1. Écrivez une fonction permettant de créer par allocation une structure de type `Monomial`.

   ```
   extern Monomial *monomial_create_full(
     const unsigned int degree,
   ```

1. https://fr.wikipedia.org/wiki/Monôme_(mathématiques)

```
  const int numerator,
  const unsigned int denominator
);
```

Au retour de l'appel de cette fonction, le monôme pointé aura

— son degré égal à `degree` ;
— sa fraction égale à la version réduite de `numerator/denominator`.

2. Écrivez une fonction permettant de remplir un monôme déjà créé avec de nouvelles valeurs.

```
extern Monomial *monomial_fill_full(
  Monomial * monomial,
  const unsigned int degree,
  const int numerator,
  const unsigned int denominator
);
```

3. Écrivez une fonction permettant de créer par allocation une structure de type `Monomial` par défaut.

```
extern Monomial *monomial_create_default(void);
```

Au retour de l'appel de cette fonction, le monôme pointé aura

— son degré égal à `monomial_degree_default` (une variable globale définie dans `monomial.c` et déclarée dans `monomial.h`) ;
— son numérateur égal à `fraction_numerator_default` ;
— son dénominateur égal à `fraction_denominator_default`.

4. Écrivez une fonction permettant de libérer toute la mémoire utilisée par un monôme.

```
extern void monomial_destroy(Monomial * monomial);
```

5. Écrivez un programme de test `test-create-destroy` permettant de valider les quatre fonctions écrites.



Exercice 4 *(Fonctions de copie et de clonage des monômes)*

1. Écrivez deux fonctions permettant d'effectuer une copie et le clonage d'un monôme.

```
extern Monomial *monomial_copy(Monomial * dest, const Monomial *
↪  src);
extern Monomial *monomial_clone(const Monomial * src);
```

2. Écrivez deux programmes de test `test-copy` et `test-clone` permettant de valider les deux fonctions écrites.



Exercice 5 *(Fonctions d'accès aux données des monômes)*

1. Écrivez trois fonctions permettant de récupérer les valeurs du degré, numérateur et dénominateur d'un monôme.

```
extern unsigned int monomial_get_degree(const Monomial * monomial);
extern int monomial_get_numerator(const Monomial * monomial);
extern unsigned int monomial_get_denominator(const Monomial *
↪  monomial);
```

Écrivez deux fonctions permettant de modifier les valeurs du degré, numérateur et dénominateur d'un monôme.

© 2011-2023 Christophe Demko

```
extern Monomial *monomial_set_degree(
  Monomial * monomial,
  const unsigned int degree
);
extern Monomial *monomial_set_fraction(
  Monomial * monomial,
  const int numerator,
  const unsigned int denominator
);
```

2. Écrivez un programme de test `test-accessors` permettant de valider les 5 fonctions écrites.



Exercice 6 *(Conversion en chaine de caractères des monômes)*

1. Écrivez une fonction permettant de convertir un monôme en chaine de caractères.

   ```
   extern const char *monomial_to_string(const Monomial * monomial);
   ```

   Il devra être possible d'effectuer

   ```
   printf("%s\n", monomial_to_string(monomial));
   ```

   sans perte de mémoire. Le résultat sera affiché sous la forme `5:-1/8`,

2. Écrivez un programme de test `test-to-string` permettant de valider la fonction écrite.



Exercice 7 *(Écriture et lecture sur un fichier des monômes)*

1. Écrivez deux fonctions permettant d'écrire et de lire sur un fichier déjà ouvert.

   ```
   extern Monomial * monomial_fwrite(
     const Monomial * monomial,
     FILE * stream
   );
   extern Monomial * monomial_fread(
     Monomial * monomial,
     FILE * stream
   );
   ```

2. Écrivez un programme de test `test-file` permettant de valider les 2 fonctions écrites.

2 Polynômes univariés

Un polynôme [2] univarié est un polynôme défini sur une seule variable. Ils seront représentés par un pointeurs vers des monômes affublés d'un degré.

Le polynôme $\frac{1}{5}x + \frac{2}{3}x^3 - \frac{1}{8}x^5$ sera représenté par la figure 2.

2. https://fr.wikipedia.org/wiki/Polynôme

© 2011-2023 Christophe Demko

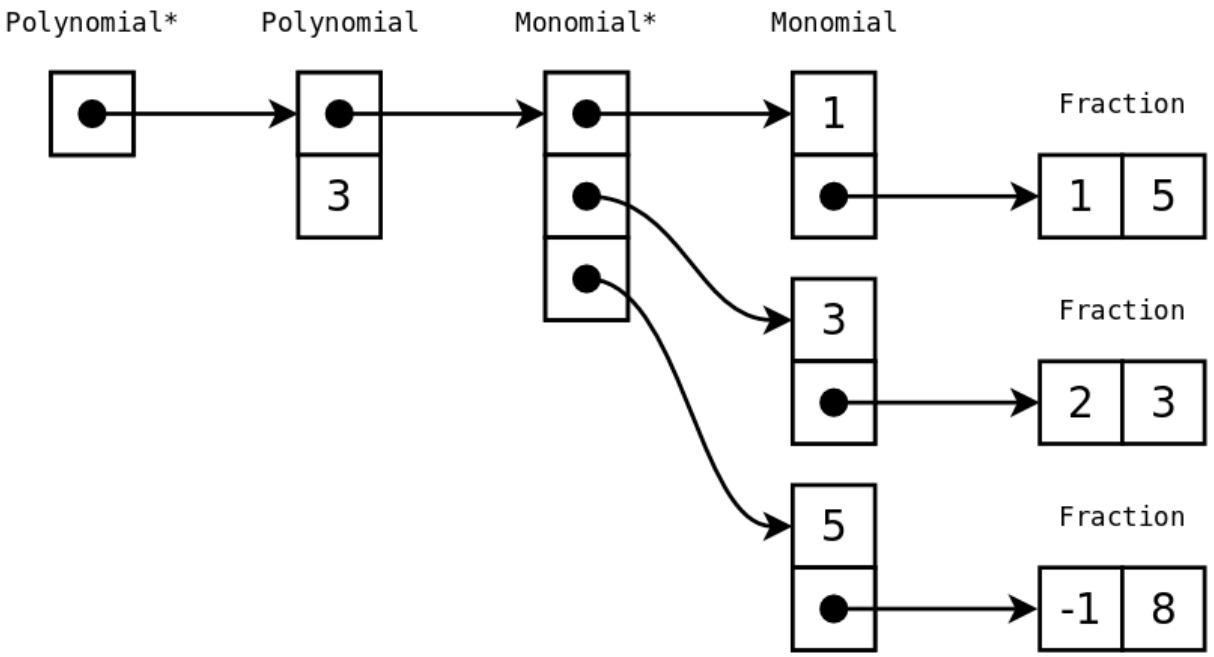


FIGURE 2 – Représentation d'un polynôme



Exercice 8 *(Définition du type `Polynomial`)*
Dans un nouveau projet, définissez le type `Polynomial` permettant de stocker un polynôme.

```
typedef struct _Polynomial Polynomial;
```

Faites en sorte d'encapsuler la structure dans un fichier `polynomial.inc`. Il ne doit pas y avoir de structures dans le fichier `polynomial.h`.



Exercice 9 *(Fonctions d'initialisation et de finalisation des polynômes)*

1. Écrivez deux fonctions permettant d'initialiser (resp. de finaliser) la librairie gérant les polynômes.

   ```
   extern bool polynomial_init(void);
   extern bool polynomial_finish(void);
   ```

   La fonction d'initialisation doit appeler la fonction d'initialisation déjà définie dans les monômes `monomial_init`. La fonction de finalisation doit appeler la fonction de finalisation déjà définie dans les monômes `monomial_finish`.

2. Écrivez un programme de test `test-init-finish` permettant de valider les 2 fonctions écrites.



Exercice 10 *(Fonctions de création et de destruction des polynômes)*

1. Écrivez une fonction permettant de créer par allocation une structure de type `Polynomial`.

   ```
   extern Polynomial *polynomial_create(void);
   ```

   Au retour de l'appel de cette fonction, le polynôme pointé ne contiendra aucun monôme.

2. Écrivez une fonction permettant de réinitialiser un polynôme.

   ```
   extern Polynomial *polynomial_erase(Polynomial * polynomial);
   ```

   Au retour de l'appel de cette fonction, le polynôme ne contiendra plus aucun monôme.

© 2011-2023 Christophe Demko

3. Écrivez une fonction permettant de libérer toute la mémoire utilisée par un polynôme.

   ```
   extern void polynomial_destroy(Polynomial * polynomial);
   ```

   Écrivez un programme de test `test-create-destroy` permettant de valider les trois fonctions écrites.



Exercice 11 *(Fonctions de copie et de clonage des polynômes)*

1. Écrivez deux fonctions permettant d'effectuer une copie et le clonage d'un polynôme.

   ```
   extern Polynomial *polynomial_copy(
     Polynomial * dest,
     const Polynomial * src
   );
   extern Polynomial *polynomial_clone(
     const Polynomial * src
   );
   ```

2. Écrivez deux programmes de test `test-copy` et `test-clone` permettant de valider les deux fonctions écrites.



Exercice 12 *(Fonctions d'accès aux données des polynômes)*

1. Écrivez une fonction permettant de spécifier à un polynôme le numérateur et le dénominateur pour un degré donné. Dans cette fonction, le dénominateur peut ne pas être spécifié si le numérateur est nul.

   ```
   extern Polynomial *polynomial_set_full(
     Polynomial * polynomial,
     const unsigned int degree,
     const int numerator,
     const unsigned int denominator
   );
   ```

   Au retour de l'appel de cette fonction, le polynôme aura été mis à jour en respectant les contraintes suivantes :

   — si le numérateur passé en argument est nul, le polynôme ne contient plus de données pour ce degré ;
   — si le degré n'existait pas avant l'opération, un nouveau monôme a été créé avec sa fraction associée ;
   — si le degré existait déjà avant l'opération, sa fraction a été mise à jour ;
   — les monômes sont maintenus triés par degré croissant.

2. Écrivez deux fonctions permettant de récupérer le numérateur et le dénominateur d'un polynôme pour un degré donné.

   ```
   extern int polynomial_get_numerator(
     const Polynomial * polynomial,
     const unsigned int degree
   );
   extern unsigned int polynomial_get_denominator(
     const Polynomial * polynomial,
     const unsigned int degree
   );
   ```

3. Écrivez un programme de test `test-accessors` permettant de valider les trois fonctions écrites.



Exercice 13 *(Fonction de conversion en chaine de caractères des polynômes)*

1. Écrivez une fonction permettant de transformer un polynôme en chaine de caractères.

   ```
   extern const char *polynomial_to_string(const Polynomial *
   ↪  polynomial);
   ```

   Il devra être possible d'effectuer

   ```
   printf("%s\n", polynomial_to_string(polynomial));
   ```

   sans perte de mémoire. Le résultat devra être de la forme `{1:3/5,3:2/3,5:-1/8}`

2. Écrivez un programme de test `test-to-string` permettant de valider la fonction écrite.



Exercice 14 *(Écriture et lecture sur un fichier des polynômes)*

1. Écrivez deux fonctions permettant d'écrire et de lire sur un fichier déjà ouvert.

   ```
   extern Polynomial * polynomial_fwrite(
     const Polynomial * polynomial,
     FILE * stream
   );
   extern Polynomial * polynomial_fread(
     Polynomial * polynomial,
     FILE * stream
   );
   ```

2. Écrivez un programme de test `test-file` permettant de valider les 2 fonctions écrites.

3 Itérateurs sur les polynômes

Un polynôme étant une collection de monôme, il peut être intéressant d'essayer d'implémenter le *patron* de programmation *Itérateur* [3].

```
#include <stdio.h>
#include <stdlib.h>

#include "./polynomial.h"

int main(void) {
  polynomial_init();
  {
    PolynomialIterator *iterator;
    Polynomial *p = polynomial_create();
    p = polynomial_set_full(p, 1, 3, 5);
    p = polynomial_set_full(p, 3, 2, 3);
    p = polynomial_set_full(p, 5, -1, 8);
```

3. https://fr.wikipedia.org/wiki/Itérateur

```
    for (iterator = polynomial_iterator_create(p);
         polynomial_iterator_valid(iterator);
         iterator = polynomial_iterator_next(iterator)) {
      printf("%u:%d/%u\n", polynomial_iterator_get_degree(iterator),
             polynomial_iterator_get_numerator(iterator),
             polynomial_iterator_get_denominator(iterator));
    }
    polynomial_iterator_destroy(iterator);
    polynomial_destroy(p);
  }
  polynomial_finish();
  return EXIT_SUCCESS;
}
```

Ce code permet d'afficher les lignes suivantes :

```
1:3/5
3:2/3
5:-1/8
```



Exercice 15 *(Définition du type `PolynomialIterator`)*
Définissez un nouveau type `PolynomialIterator` dans les fichiers `polynomial.c` et `polynomial.inc` (en utilisant la convention maintenant classique d'encapsulation) permettant d'itérer sur un polynôme. Ce type contiendra un pointeur vers le polynôme itéré et un entier indiquant quel monôme l'itérateur a atteint.



Exercice 16 *(Fonctions de création et de destruction des itérateurs de polynômes)*
Écrivez deux fonctions

```
extern PolynomialIterator * polynomial_iterator_create(
  const Polynomial * polynomial
);
extern void polynomial_iterator_destroy(
  PolynomialIterator * iterator
);
```

permettant de créer un itérateur et de libérer sa mémoire.



Exercice 17 *(Fonctions d'itération sur les polynômes)*
Écrivez trois fonctions

```
extern PolynomialIterator *polynomial_iterator_next(
  PolynomialIterator * iterator
);
extern PolynomialIterator *polynomial_iterator_rewind(
  PolynomialIterator * iterator
);
extern bool polynomial_iterator_valid(
  const PolynomialIterator * iterator
);
```

© 2011-2023 Christophe Demko

permettant d'itérer à partir d'un itérateur.

- `polynomial_iterator_next` permet de faire avancer l'itérateur au prochain monôme ;
- `polynomial_iterator_rewind` permet de réinitialiser l'itérateur sur le premier monôme ;
- `polynomial_iterator_valid` permet de savoir s'il reste encore des éléments sur lesquels itérer dans l'itérateur ;



Exercice 18 *(Fonctions d'accès aux données sur les itérateurs de polynômes)*

1. Écrivez trois fonctions

```
extern unsigned int polynomial_iterator_get_degree(
  const PolynomialIterator * iterator
);
extern int polynomial_iterator_get_numerator(
  const PolynomialIterator * iterator
);
extern unsigned int polynomial_iterator_get_denominator(
  const PolynomialIterator * iterator
);
```

   permettant d'accéder aux données du monôme courant d'un itérateur.

   - `polynomial_iterator_get_degree` permet de récupérer le degré du monôme courant ;
   - `polynomial_iterator_get_numerator` permet de récupérer le numérateur du monôme courant ;
   - `polynomial_iterator_get_denominator` permet de récupérer le dénominateur du monôme courant ;

2. Écrivez un programme de test `test-iterate` permettant de valider les fonctions écrites sur les itérateurs.

4 Opérations sur les polynômes

Les opérations d'addition et de multiplication peuvent être définies sur les polynômes. Vous chercherez sur internet leurs définitions exactes.



Exercice 19 *(Addition de polynômes)*

1. Écrivez une fonction permettant d'additionner deux polynômes.

```
extern Polynomial *polynomial_addition(
  Polynomial * polynomial,
  const Polynomial * a,
  const Polynomial * b
);
```

   Le calcul de l'addition de a et b est placé dans `polynomial`. Le résultat retourné est également le résultat du calcul.

2. Écrivez un programme de test `test-addition` permettant de valider la fonction écrite.

© 2011-2023 Christophe Demko



Exercice 20 *(Multiplication de polynômes)*

1. Écrivez une fonction permettant de multiplier deux polynômes.

   ```
   extern Polynomial *polynomial_multiplication(
     Polynomial * polynomial,
     const Polynomial * a,
     const Polynomial * b
   );
   ```

   Le calcul de la multiplication de a et b est placé dans `polynomial`. Le résultat retourné est également le résultat du calcul.

2. Écrivez un programme de test `test-multiplication` permettant de valider la fonction écrite.

5 Polynômes multivariés

Un polynôme multivarié est un polynôme défini sur plusieurs variables. Ils seront représentés par un tableau de monômes multivariés affublés d'un degré.

Le polynôme $p(x, y, z) = \frac{3}{5}y^3z - \frac{1}{8}x^2z^4$ sera représenté par la figure 3

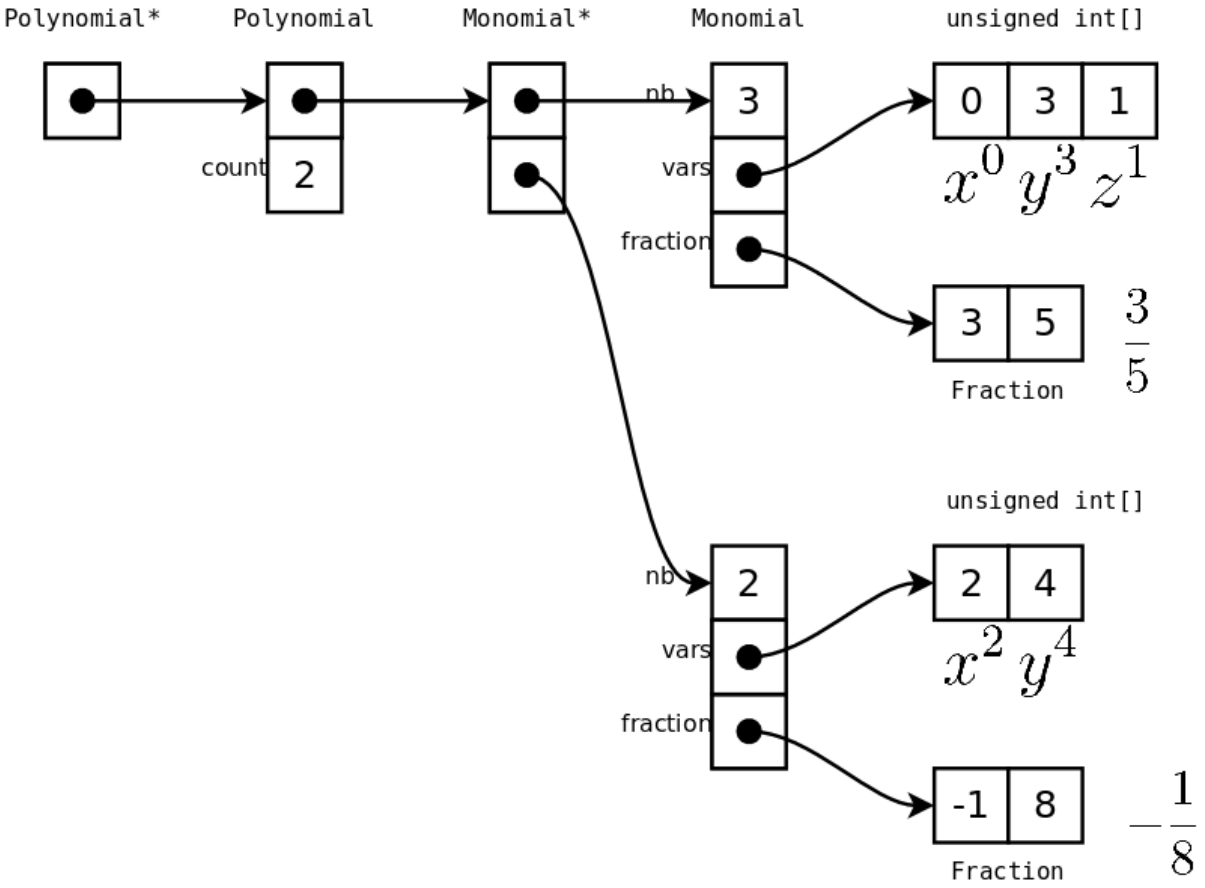


FIGURE 3 – Représentation d'un polynôme multivarié



Exercice 21 *(Revisiter les fonctions sur les polynômes multivariés)*
Trouvez le moyen de revisiter les fonctions précédentes pour pouvoir gérer les polynômes multivariés.

Historique des modifications

2011-2012_1 *Lundi 13 février 2012*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Création initiale du document

2011-2012_2 *Jeudi 23 février 2012*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Correction d'une erreur dans le schéma des polynômes.

2012-2013_1 *Mardi 22 janvier 2013*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Mise à jour.

2012-2013_2 *Mardi 12 février 2013*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Correction de coquilles.

2012-2013_3 *Mardi 5 mars 2013*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Déplacement des exercices de `load et unload`.

2012-2013_4 *Mardi 12 mars 2013*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Correction de coquilles.

2012-2013_5 *Samedi 16 mars 2013*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement de titre et sous-titre.

2013-2014_1 *Lundi 10 février 2014*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Ajout de `cmake` et `codeblocks`.

2013-2014_2 *Lundi 19 février 2014*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Retrait d'un doublon `const`.

2014-2015_1 *Vendredi 6 février 2015*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement d'année.

2014-2015_2 *Mercredi 18 février 2015*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

© 2011-2023 Christophe Demko

— Correction de coquilles ;
— Effacement du paragraphe concernant la variable `Path` de Window$.

2015-2016_1 *Mercredi 27 janvier 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Passage à *pandoc* ;
— retrait de `cmake` et `codeblocks`.

2015-2016_2 *Mercredi 3 février 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Retrait du mot-clé `const` pour les fonctions *setter* de la librairie monomial.

2015-2016_3 *Dimanche 7 février 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Utilisation du moteur *pdflatex* permettant de ne pas imposer d'espace après les caractères `:`.

2015-2016_4 *Mardi 23 février 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Renommage du fichier de test pour la création et la destruction des itérateurs.

2015-2016_5 *Mardi 23 février 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Précision de la définition du type `PolynomialIterator` ;
— Regroupement des programmes de test sur les itérateurs.

2015-2016_6 *Mercredi 24 février 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement du retour de la fonction `polynomial_iterator_valid`.

2015-2016_7 *Dimanche 28 février 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Réinsertion du *copyright* qui avait disparu lors du passage à la nouvelle version de *pandoc*.

2015-2016_8 *Lundi 6 juin 2016*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Ajout d'icônes.

2016-2017_1 *Samedi 4 février 2017*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement d'année.

2016-2017_2 *Vendredi 17 février 2017*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Corrections orthographiques.

2016-2017_3 *Lundi 13 mars 2017*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Mise à jour du copyright.

© 2011-2023 Christophe Demko

2017-2018_1 *Mardi 30 janvier 2018*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement d'année.

2017-2018_2 *Lundi 5 février 2018*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Utilisation du terme *finalisation* plutôt que *terminaison*.

2017-2018_3 *Lundi 12 février 2018*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Précision dans les fonctions *finalisation* et *initialisation* ;

— Correction des adresses email.

2018-2019_1 *Jeudi 15 novembre 2018*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement d'année ;

— Changement de l'indentation.

2019-2020_1 *Lundi 14 octobre 2019*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Utilisation du template https://github.com/Wandmalfarbe/pandoc-latex-template.

2020-2021_1 *Vendredi 9 octobre 2020*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement d'année ;

— Changement dans les représentations de polynômes.

2020-2021_2 *Vendredi 9 octobre 2020*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Correction de coquilles.

2020-2021_3 *Dimanche 11 octobre 2020*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Ajout de couleurs sur la page de titre.

2020-2021_4 *Dimanche 18 octobre 2020*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Regroupement de la fonction `polynomial_erase` avec les fonctions de création et de destruction ;

— Changement de la sortie de la fonction `polynomial_to_string` ;

— Retrait de l'argument optionnel de `polynomial_set_full`.

2021-2022_1 *Mercredi 6 octobre 2021*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Changement d'année.

2022-2023_1 *Jeudi 19 janvier 2023*

Dr Christophe Demko <christophe.demko@univ-lr.fr>

— Transformation du cahier initial en cahier de complément pour les enseignements "Programmation impérative".